

Lab 3: Full Step-by-Step Guide

Day 3: React Fundamentals

React Development: Beginner to Professional · Delegate Handbook · Capstone project: TaskBoard

> Objective: Build user interfaces from reusable, composable components.

Contents

? to use this handbook](#how-to-use-this-handbook)
? 3 at a glance](#day-3-at-a-glance)
? ning recap](#morning-recap)
? ule 3.1: JSX](#module-31-jsx)
? ule 3.2: Components and Props](#module-32-components-and-props)
? ule 3.3: Rendering Data](#module-33-rendering-data)
? ule 3.4: Thinking in React](#module-34-thinking-in-react)
? ds-on labs](#hands-on-labs)
? 3.1: Build a Reusable Component Set](#lab-31-build-a-reusable-component-set)
? 3.2: Render a Product Catalogue from Static Data](#lab-32-render-a-product-catalogue-from-static-data)
? 3.3: TaskBoard: Static Task List UI](#lab-33-taskboard-static-task-list-ui)
? wledge check](#knowledge-check)
? takeaways](#key-takeaways)
? ther reading](#further-reading)
? wer key](#answer-key)

How to use this handbook

This handbook accompanies the instructor-led session. It follows the same order as the slides, explains each concept in plain language, and gives you everything you need to complete the labs and revise afterwards.

? **Modules** explain each topic, with code examples you can type and run.
? **Code** appears in code blocks, with the file name in bold above it. Type it yourself rather than copying: it builds memory and teaches you to read errors.
? **Callouts** marked Tip, Good to know, Try it, Troubleshooting and Check your understanding highlight key ideas. Answers are hidden in expandable sections so you can test yourself first.
? **Labs** have a goal, numbered steps and a *Done when* checklist.
? **The answer key** at the back covers the morning recap and the knowledge check.

Day 3 at a glance

Part	Topic
---	---
Recap	Day 2 knowledge check
3.1	JSX: writing UI in JavaScript

- | 3.2 | Components and props |
- | 3.3 | Rendering lists and conditional content |
- | 3.4 | Thinking in React and React DevTools |
- | Labs | Component set, product catalogue, TaskBoard list |

Each module builds directly on the one before. JSX is the language of components. Components receive data through props. Lists and conditions turn that data into dynamic screens. Finally, you learn how professionals plan a React UI before writing it. By this afternoon TaskBoard will display a real board of tasks, built entirely from components you wrote.

By the end of today you will be able to

- ? Write JSX that embeds JavaScript expressions, and explain how JSX differs from HTML (`className``, `htmlFor``, closed tags, camelCase, style objects, comments)
- ? Group elements with a Fragment instead of an extra `div``
- ? Create function components that receive props, use default values and render `children``
- ? Compose small components into a tree where data flows down through props
- ? Render arrays with `map``, choose good keys, and show or hide content with early returns, ternaries and `&&``, avoiding the `0 &&`` trap
- ? Plan a UI with Thinking in React, organise a project's folders and files, and inspect components with React Developer Tools

Labs today

- | Lab | Title | Time |
|-----|---|--------|
| --- | --- | --- |
| 3.1 | Build a Reusable Component Set | 40 min |
| 3.2 | Render a Product Catalogue from Static Data | 40 min |
| 3.3 | TaskBoard: Static Task List UI | 45 min |

> Before you start

- > - Install the React Developer Tools browser extension. Search "React Developer Tools" in the Chrome Web Store, Edge Add-ons or Firefox Add-ons. You will use it from this morning onwards.
- > - Open your taskboard project in VS Code and start the dev server with `npm run dev`. Keep it running all day, and use a second terminal tab for Git commands.

Morning recap

Answer these questions on Day 2 before the session starts. The answers are in the [answer key](#answer-key).

1. What does `const double = (n) => n * 2` return for `double(5)`?
2. Write the immutable way to set `done` to `true` on a task object.
3. Which array method keeps only items that pass a test?
4. Why must you check `res.ok` after `fetch`?
5. Which file mounts the React app into the page?

Last night's take-home practice asked you to add an array of three tasks to TaskBoard's `App.jsx`, above the component. If you have not done it yet, add it now: today's examples use it.

`src/App.jsx` (above function `App()`)

```
const tasks = [
```

```
{ id: 1, title: "Set up project", done: true },
{ id: 2, title: "Build task list", done: false },
{ id: 3, title: "Style the board", done: false },
];
---
```

Module 3.1: JSX

HTML-like syntax that is really JavaScript.

JSX looks like HTML, but it is JavaScript underneath. Vite converts it into regular JavaScript function calls before the browser sees it. Because it is JavaScript, it has a few rules that differ from HTML.

Embedding JavaScript with curly braces

```
src/App.jsx

function App() {
  const user = { name: "Zanele", role: "Developer" };
  const openTasks = 4;
  const today = new Date().toLocaleDateString();
  return (
    Welcome back, {user.name}
    Role: {user.role.toUpperCase()}
    {openTasks} open tasks
    Points: {openTasks * 2}
    Today is {today}
  );
}
export default App;
```

Inside JSX, curly braces open a window into JavaScript. Whatever expression you put inside is evaluated, and its result is shown on screen.

An expression is anything that produces a value: a variable, maths, a method call, a ternary. Statements such as if, for and const declarations are not allowed inside the braces. Do that logic above the return statement, as the example does with today.

When JSX spans multiple lines, wrap it in round brackets after return. Without them, JavaScript can insert a semicolon straight after return, and your component returns nothing.

Key points:

- ? `{}` switches from markup to JavaScript.
- ? Any expression works: variables, maths, function calls.
- ? Statements do not: no `if` or `for` inside `{}`.
- ? Wrap multi-line JSX in `()`.

> Try it: type the example into App.jsx and change the name: the browser updates as soon as you save. Then add `{if (openTasks > 0) "busy"}` inside main and read the syntax error. Fix it with a ternary, which *is* an expression: `{openTasks > 0 ? "Busy day" : "All clear"}`.

> Check your understanding

> Can you put {user} directly in JSX?

>

> Answer

>

> No. Objects cannot be rendered; React throws Objects are not valid as a React child. Render a property such as user.name instead.

>

>

JSX is not quite HTML

```
| HTML | JSX |
| --- | --- |
| | |
| Email | Email |
| | |
| | |
| | |
| | |
| | { /* comment */ } |
```

There are five differences to learn:

1. class becomes className, because class is a reserved word in JavaScript. for on labels becomes htmlFor for the same reason.
2. Every tag must be closed. Empty elements such as input, img and br need a self-closing slash.
3. Attributes use camelCase. onclick becomes onClick, tabIndex becomes tabIndex. The exceptions are aria-* and data-* attributes, which keep their dashes, for example aria-label.
4. Event handlers receive a function in curly braces, not a string. Notice onClick={save}, not onClick={save()}. With brackets, save would run immediately during rendering instead of on click. This is a very common bug.
5. style takes a JavaScript object, so you see double curly braces: the outer pair means "JavaScript here", the inner pair is the object. CSS property names become camelCase, and a plain number means pixels for sizes such as fontSize: 14. Unitless properties such as lineHeight, opacity and zIndex keep the plain number.

Comments inside JSX use curly braces around a block comment. In VS Code, Ctrl+/ (Windows) or Cmd+/ (macOS) inserts the right kind automatically.

> Tip

> If you paste HTML from elsewhere, search for "HTML to JSX converter" online, or let VS Code's error highlights guide you.

One parent element, and Fragments

src/App.jsx

// Error: two sibling elements returned

```
function Broken() {
  return (
    TaskBoard
    Plan your work
  )
}
```

```

);
}

// Fix: wrap in a Fragment (adds nothing to the page)
function Header() {
return (
<>
TaskBoard
Plan your work
);
}

```

A component must return a single top-level element. Returning two siblings is like a function returning two values; JavaScript cannot do that.

You could wrap them in a div, but that adds an extra element to the page for no reason, which can break CSS layouts such as Flexbox and Grid. Instead, use a Fragment, written as empty angle brackets `<>` and `.` It groups the elements without adding anything to the HTML.

If a wrapper element has meaning, such as header, section or article, use it. Use a Fragment when you just need grouping.

Key points:

- ? A component returns one element.
- ? Fragments `<> </>` group without adding extra HTML.
- ? Use a real element such as `header` or `section` when it adds meaning.
- ? Fragments keep the page free of unnecessary `div`'s.

> Try it: paste the Broken component into App.jsx and read the error: Adjacent JSX elements must be wrapped in an enclosing tag. Fix it with a Fragment, then open DevTools Elements and confirm no extra element was added to the page.

> Good to know: the long form

> `<Fragment>`, imported from `react`, is the same thing. It is needed when you must give the fragment a key, for example when `map` returns a group of sibling elements for each item without a wrapper. Current React also lets `take a ref`, which is an advanced feature.

Module 3.2: Components and Props

Reusable building blocks with inputs.

Props are how components receive data. If a component is a function, props are its arguments. This module is the heart of React.

Creating and using a component

```

src/components/TaskCard.jsx

function TaskCard({ title, assignee, points, done }) {
return (
{title}
Assigned to {assignee}
{points} pts

```

```

Status: {done ? "Done" : "Open"}
);
}
export default TaskCard;

src/App.jsx

import TaskCard from "../components/TaskCard.jsx";

// inside App's return:
<TaskCard title="Design login" assignee="Naledi"
points={3} done={false} />

```

Create a folder `src/components` and a file `TaskCard.jsx`. The convention is one component per file, with the file named after the component.

`TaskCard` receives a props object. We destructure `title`, `assignee`, `points` and `done` straight away, exactly as you practised on Day 2.

To use it, import it and write it like an HTML tag. Pass props like attributes:

? **Strings** can use quotes.

? **Numbers, booleans, arrays, objects and functions** must use curly braces. ``points="3"`` would be the string ``"3"`,` not the number 3.

Props are read-only. A component must never change its own props. If data needs to change, that is *state*, which we cover tomorrow.

Key points:

? One component per file, named with a capital letter.

? Props arrive as one object, destructured in the parameters.

? Strings use quotes; everything else uses `{ }`.

? Props are read-only inside the component.

> Try it: render three `TaskCards` in `App` with different props. Then try the boolean shortcut: writing just `done` is the same as `done={true}`.

> Check your understanding

> What happens if you forget to pass `assignee`?

>

> Answer

>

> It is undefined, so the paragraph shows "Assigned to" with nothing after it. We fix that next with defaults.

>

>

> Troubleshooting

>

> | Problem | Fix |

> | --- | --- |

> | `TaskCard` is not defined | The import is missing or the path is wrong. VS Code can add the import for you: type `<TaskCard`, then press `Ctrl+Space` or accept the suggestion. |

Default values and the children prop

src/components/ui/Card.jsx

```
function Card({ title, variant = "default", children }) {  
  return (  
    {title && {title}}  
    {children}  
  );  
}  
export default Card;
```

Usage

Ends Friday.

View board

Default values work exactly as on Day 2: `variant = "default"` applies whenever the prop is not passed.

`children` is a special prop. It contains whatever you put between the opening and closing tags of the component. Here, the paragraph and the button become children, and Card decides where to place them.

This is composition. Card does not need to know what is inside it; it just provides the frame. Modals, layouts, panels and page shells all work this way.

Notice the `className` uses a template literal, so the variant becomes part of the class: `card card-highlight`. You will style with Tailwind on Day 8, but the same idea applies.

The title line uses `&&`, which Module 3.3 covers in detail. It shows the heading only if a title was passed.

Key points:

? Defaults in destructuring handle missing props.

? ``children`` is whatever you place between the tags.

? Great for wrappers: cards, modals, layouts.

? Template literals build dynamic class names.

> Try it: build Card in `src/components/ui/Card.jsx` (you will reuse this file in Lab 3.1), import it into App with `import Card from "../components/ui/Card.jsx"`, and wrap two different pieces of content in it.

> Check your understanding

> Where would you use a component with children in TaskBoard?

>

> Answer

>

> A Column that wraps task cards, a Modal for editing a task, a Layout for every page.

>

>

Data flows down

1. Parent owns the data. App holds the list of tasks.

2. Parent passes props. App gives each TaskCard the data it needs: title, points, done.

3. Child displays props. TaskCard renders what it receives and never edits its props.

4. Changes start at the top. When data changes in App, React re-renders the children with new props.

React uses one-way data flow. Data moves down the component tree through props, like water flowing downhill. The parent owns the data and passes pieces of it to children. Children display what they receive. If the data changes, it changes in the parent, and the new values flow down again.

Why is this good? When something looks wrong on screen, you follow the props upwards to find where the data came from. There is one source of truth, which makes bugs much easier to track down.

You might be asking: how does a child, such as a checkbox in TaskCard, tell the parent to mark a task done? The parent passes a function down as a prop, and the child calls it. You will do exactly that tomorrow.

> Try it: sketch TaskBoard's tree on paper: App at the top holding the tasks array, with arrows down to Board, then Column, then TaskCard. Label each arrow with the props that component receives.

Composition: building a tree

src/components/Board.jsx

```
import Column from "../Column.jsx";

function Board({ tasks }) {
  const todo = tasks.filter((t) => !t.done);
  const done = tasks.filter((t) => t.done);
  return (

);
}
```

export default Board;

src/App.jsx (inside the return)

Real UIs are trees of components. Board takes the full task list, splits it into to-do and done using filter, and renders two Columns. Each Column will render a list of TaskCards, which you build in the next module.

Notice the filtering happens above the return. Do your data preparation in plain JavaScript first, then keep the JSX clean.

App now contains just one line of markup: Board with tasks. That is the goal: top-level components read like a table of contents.

How small should a component be? A good rule: if you need the word "and" to describe what it does, it might be two components.

Key points:

? Small components combine into bigger ones.

? `Board` splits the data; `Column` displays it.

? Derive what you need with `filter` before `return`.

? `App` stays short and readable.

> Try it: create Board.jsx now, and import it into App with import Board from "../components/Board.jsx";. Board imports Column, which you complete in the next module, so for now create a placeholder that just shows its heading and the number of tasks:

>

> src/components/Column.jsx (placeholder)

```

>
> jsx
> function Column({ heading, tasks }) {
>   return (
>
>     {heading} ({tasks.length})
>
>   );
> }
> export default Column;
>
> Check your understanding
> Which component should own the tasks array: App, Board or Column?
>
> Answer
>
> App, for now. It is the highest point that needs it. You will formalise this rule tomorrow.
>
>
---
```

Module 3.3: Rendering Data

Lists, keys and conditional content.

Almost every screen shows a list of something. This module shows how React turns an array into elements, and how to show or hide content based on data.

Rendering a list with map

```

src/components/Column.jsx

import TaskCard from "../TaskCard.jsx";

function Column({ heading, tasks }) {
  return (
    {heading} ({tasks.length})
    {tasks.map((task) => (
      ))}
  );
}

export default Column;
```

This is the most common pattern in React. Inside curly braces, call map on the array. For each task, return some JSX. React renders the resulting array of elements.

Look carefully at the arrow function: after the arrow there are round brackets, not curly braces. That is the implicit return from Day 2. If you use curly braces here, you must write return,

otherwise nothing renders. This is the number one list bug.

Every item in a list needs a key prop, placed on the outermost element returned by map. Here that is the li, not the TaskCard.

`{...task}` spreads all the properties of the task object as props. It is shorthand for `title={task.title} points={task.points}` and so on. It is useful, but be careful not to pass props the child does not need.

Key points:

? ``map`` inside ``{}`` returns an array of elements.

? Round brackets after ``=>`` give an implicit return.

? ``key`` goes on the outermost element in ``map``.

? `{...task}` spreads every property as a prop.

> Try it: replace your placeholder Column.jsx with the version above and check the browser. Open the Console and confirm there are no key warnings. Then remove `key={task.id}` and watch the warning appear: Each child in a list should have a unique "key" prop. Put the key back.

Why keys matter

? Keys identify items so React can tell which were added, removed or moved.

? Use a stable, unique id from your data: ``task.id``, ``user.email``.

? Avoid the array index when items can be reordered, inserted or deleted.

? Never generate keys during render, for example with ``Math.random()``.

? Keys only need to be unique among siblings, not globally.

When a list changes, React must work out what happened. Was an item removed? Added? Moved? Keys give each item a stable identity so React can match old and new items correctly. Use an id that comes from your data; database ids are perfect.

Using the array index as the key seems convenient, and it silences the warning, but it causes real bugs when items are reordered or removed. For example, if each item has a text input and you delete the first item, the typed text can appear on the wrong item because the indexes shifted. The index is acceptable only for static lists that never change order, such as a fixed list of menu labels.

Never use `Math.random()` or `Date.now()` as a key during render. The key changes every render, so React destroys and rebuilds every item every time. That is slow and loses anything the user typed.

> Analogy

> Keys are like name tags at a conference. If everyone swaps seats, you still know who is who. Index keys are like seat numbers: they describe the seat, not the person.

> Check your understanding

> Your data has no id. What can you do?

>

> Answer

>

> Generate one when the item is created, for example with `crypto.randomUUID()`, and store it with the item.

>

>

Conditional rendering patterns

src/components/TaskCard.jsx

```
function TaskCard({ title, points, done, assignee }) {
  if (!title) return null;      // 1. early return
  return (
    {title}
    /* 2. ternary: one or the other */
    {done ? Done : Open}
    /* 3. &&: show or show nothing */
    {assignee && Assigned to {assignee}}
    /* Careful: 0 would render as "0" */
    {points > 0 && {points} pts}
  );
}
export default TaskCard;
```

Three patterns cover nearly every case where the UI depends on data:

1. Early return. If there is nothing sensible to show, return null. React renders nothing.
2. The ternary. Use it when you want to show one thing or another. The example also uses it to choose a class name.
3. Logical AND (&&). If the left side is truthy, React renders the right side. If it is falsy, the expression gives back that falsy value, and React renders nothing for false, null or undefined. It is perfect for optional parts such as the assignee line.

The trap: React does not render false, null or undefined, but it *does* render the number 0. If points is 0, {points && ...} displays a stray 0 on screen. Always compare explicitly: points > 0 &&.

If conditions get complex, move the logic above the return into a variable, or split out a separate component. Nested ternaries inside JSX are very hard to read.

Key points:

- ? Early `return null` renders nothing.
- ? A ternary chooses between two outputs.
- ? `&&` shows something or nothing.
- ? Trap: `{count && ...}` shows `0` when `count` is 0. Compare explicitly.

> Try it: give one task points: 0 and change the last line to {points && {points} pts}. Find the lonely 0 on the card, then fix it with points > 0 &&.

> Check your understanding

> Which pattern would you use to show "No tasks yet" when a column is empty?

>

> Answer

>

> A ternary on tasks.length === 0.

>

>

Module 3.4: Thinking in React

Planning a UI like a professional.

Knowing the syntax is not enough. Professionals plan the component structure before they type. This module teaches that process, based on the official React documentation.

From design to components

1. Break the UI into a hierarchy. Draw boxes around every part. Each box is a component. Nest them.
2. Build a static version. Render the design from props only. No interactivity yet.
3. Find the minimal state. What data changes over time? Keep only that. Derive the rest.
4. Decide where state lives. Put it in the closest common parent of the components that use it.
5. Add inverse data flow. Pass functions down so children can request changes.

This five-step process comes from the [Thinking in React](https://react.dev/learn/thinking-in-react) page in the official docs. It is how experienced developers approach every new screen.

? **Step one:** take a screenshot or design and draw boxes around every part. Each box becomes a component, and boxes inside boxes become child components.

? **Step two:** build a static version using only props and hardcoded data. No clicks, no state. This is what you are doing today with TaskBoard.

? **Steps three to five** are about state and interactivity, which we cover tomorrow. Step three asks: what is the smallest set of data that changes? For example, the task list changes, but the count of done tasks does not need to be stored because it can be calculated. Step four decides where that data lives. Step five lets children ask parents to change data.

> Try it: sketch the TaskBoard target design: a header with a title and summary, and a board with To do, In progress and Done columns, with cards in each. Draw a box around every part and name each box as a component.

Organising a React project

src/

components/ reusable UI pieces

TaskCard.jsx

Column.jsx

Board.jsx

Header.jsx

ui/

Button.jsx

Card.jsx

data/

tasks.js sample data

App.jsx

main.jsx

| Convention | Example |

| --- | --- |

| Components: PascalCase | TaskCard.jsx |

| Other modules: camelCase | formatDate.js |

| One component per file, default export | export default TaskCard; |

There is no single official folder structure, but this simple layout scales well for a small to medium app.

? **components** holds your building blocks. A **ui** subfolder holds generic, style-only

components such as `Button` and `Card` that know nothing about tasks.

? `**data**` holds sample data for now. On Day 7 it will be replaced by an API.

? On Day 6 you will add pages and routing, and the structure will grow a `pages` folder.

Naming conventions matter because teams rely on them. Components use PascalCase, where every word starts with a capital. Other files use camelCase.

> Try it: move your tasks array out of App.jsx into src/data/tasks.js as a named export, then import it into App:

```
>
> src/data/tasks.js
>
> js
> export const tasks = [
>   { id: 1, title: "Set up project", done: true },
>   { id: 2, title: "Build task list", done: false },
>   { id: 3, title: "Style the board", done: false },
> ];
>
>
> src/App.jsx (at the top)
>
> jsx
> import { tasks } from "../data/tasks.js";
>
```

React Developer Tools

The React Developer Tools extension (for Chrome, Edge and Firefox) adds a Components panel to the browser's DevTools. It shows your app as a tree of React components rather than HTML elements.

> Tip: inspect your components

> Open DevTools (F12 on Windows, Cmd+Option+I on macOS), find the Components tab, and click a component to see its props. Use it now to inspect TaskCard. When a card shows the wrong thing, this is the fastest way to check what data it actually received.

Hands-on labs

Work through each lab in order. Read the goal first, follow the numbered steps, and use the **Done when** checklist to confirm you have finished. Hints and troubleshooting notes follow each lab: try on your own first, then use them if you are stuck for more than a few minutes.

Lab 3.1: Build a Reusable Component Set

| Goal | Create generic Button and Card components that use props, defaults and children. |

| Suggested time | 40 min |

You are building your own mini design system. These two components will be reused throughout TaskBoard.

Steps

1. Create `src/components/ui/Button.jsx`.
2. Give it the props `variant` ("primary" or "secondary", default "primary"), `size` and `children`.
3. Build the `className` from the props with a template literal.
4. Create `src/components/ui/Card.jsx` with `title` and `children` (if you built Card during Module 3.2, you already have it).
5. Add CSS for `.btn-primary`, `.btn-secondary` and `.card` in `src/App.css`. If you removed the stylesheet import on Day 2, add `import './App.css'`; back at the top of `App.jsx`.
6. Render a Card containing two Buttons in App.
7. Inspect both components in React DevTools.
8. Commit: `git add .` then `git commit -m "Add Button and Card UI components"`.

Done when

? `Button` renders both variants

? `Card` shows a title only when one is passed

? There are no console warnings

Hints (try on your own first)

`src/components/ui/Button.jsx`

```
function Button({ variant = "primary", size = "md", children }) {  
  return {children};  
}  
export default Button;
```

`src/App.jsx`

```
import './App.css';  
import Button from './components/ui/Button.jsx';  
import Card from './components/ui/Card.jsx';  
  
function App() {  
  return (  

```

Save

Cancel

This card has no title.

```
);  
}  
export default App;
```

Troubleshooting

| Problem | Fix |

| --- | --- |

| Styles not applied | App.css is not imported in App.jsx, or the class names do not match the CSS. |

| Objects are not valid as a React child | You passed an object as children, for example {task}.

Render a property such as `{task.title}` instead. |

| Component renders nothing | The function has braces but no return. |

| Whole page blank; the Console says the module does not provide an export named 'default' | The component file is missing export default. |

> Stretch challenge

> Accept any extra props with rest, and spread them onto the button element so that `onClick` and `disabled` work automatically:

>

> `jsx`

> `function Button({ variant = "primary", size = "md", children, ...rest }) {`

> `return (`

>

> `{children}`

>

> `);`

> `}`

>

Lab 3.2: Render a Product Catalogue from Static Data

| | |

| --- | --- |

| Goal | Turn an array of objects into a grid of components with keys and conditions. |

| Suggested time | 40 min |

This lab practises lists and conditionals on a different domain, so you can see the pattern is universal. You may build it inside TaskBoard in a temporary `src/components/catalogue` folder, or in a second Vite project. The temporary folder is quicker.

Steps

1. In TaskBoard, create `src/components/catalogue/products.js` exporting an array of 8 products, each with an `id`, `name`, `price`, `category`, `inStock` and `rating`. You can use your own, or paste this in to get straight to the rendering:

`js`

`export const products = [`

`{ id: 1, name: "Wireless Mouse", price: 349, category: "electronics", inStock: true, rating: 4 },`

`{ id: 2, name: "Mechanical Keyboard", price: 1299, category: "electronics", inStock: true, rating: 5 },`
`],`

`{ id: 3, name: "USB-C Hub", price: 599, category: "electronics", inStock: false, rating: 3 },`

`{ id: 4, name: "Learning React", price: 489, category: "books", inStock: true, rating: 5 },`

`{ id: 5, name: "JavaScript Pocket Guide", price: 199, category: "books", inStock: true, rating: 0 },`

`{ id: 6, name: "Desk Lamp", price: 425, category: "office", inStock: false, rating: 4 },`

`{ id: 7, name: "Notebook Set", price: 89, category: "office", inStock: true, rating: 3 },`

`{ id: 8, name: "Laptop Stand", price: 749, category: "office", inStock: true, rating: 4 },`

`];`

Product 5's rating: 0 is deliberate — keep it, because it surfaces a common bug when you render the rating in step 2 below.

2. Create ProductCard.jsx showing the name, price and rating.
3. Show an Out of stock badge only when inStock is false.
4. Create ProductGrid.jsx that maps products to ProductCards with keys, and render it in App for now.
5. Show a No products message when the array is empty. Test it by temporarily passing an empty array (products=[]), then restore the real data.
6. Style the grid with CSS Grid.
7. Commit your work. When you move on to Lab 3.3, remove ProductGrid from App.

Done when

- ? 8 cards render in a grid
- ? Out of stock badges appear on the right products
- ? An empty array shows the message

Hints (try on your own first)

Format a price as South African rand:

```
new Intl.NumberFormat("en-ZA", { style: "currency", currency: "ZAR" }).format(price)
```

Show a whole-number rating as stars:

```
"★".repeat(rating)
```

Troubleshooting

| Problem | Fix |

| --- | --- |

| Key warning | The key is inside ProductCard (on the element it returns) rather than on the element returned by map: . |

| Stray 0 on screen | {product.rating && ...} when a rating is 0. Compare explicitly: product.rating > 0 &&. |

| Grid shows one column | display: grid is missing, or it is on the wrong element (it belongs on the container around the cards). |

> Stretch challenge

> - Add a category heading above each group of products by filtering per category.

> - Sort by price with [...products].sort((a, b) => a.price - b.price). Copy with spread first, because sort changes the array in place.

Lab 3.3: TaskBoard: Static Task List UI

| | |

| --- | --- |

| Goal | Render the TaskBoard board from mock data using a clean component tree. |

| Suggested time | 45 min |

This is your capstone for today. Notice that done has changed to status with three values, so the board can have three columns. Update the data, and any component that still reads done: Board now filters on status, and TaskCard no longer needs a Done or Open label because the column shows it. Use string ids such as "1"; they match the ids crypto.randomUUID() creates tomorrow and the Task type on Day 6.

Keep App.jsx tiny. All logic belongs in the components.

Steps

1. Put 8 sample tasks in src/data/tasks.js.
2. Give each task an id, title, assignee, points and status ("todo", "doing" or "done"). Leave assignee out of at least one task.
3. Create src/components/Header.jsx showing the app name and the total task count.
4. Update Board.jsx so it filters tasks into three Columns by status.
5. Make sure Column.jsx maps tasks to TaskCards with keys.
6. Update TaskCard.jsx to show the title, assignee and points, hiding the assignee line when it is missing.
7. Show Nothing here yet in an empty column.
8. Style the board as three columns in App.css, then commit and push:

bash

git add .

git commit -m "Build static TaskBoard UI"

git push

Done when

- ? Three columns show the correct tasks
- ? The counts in the column headings are correct
- ? `App.jsx` renders only `Header` and `Board`

Hints (try on your own first)

Data shape (src/data/tasks.js; add five more tasks of your own)

```
export const tasks = [  
  { id: "1", title: "Set up project", assignee: "Zanele", points: 2, status: "done" },  
  { id: "2", title: "Build task list", assignee: "Sipho", points: 5, status: "doing" },  
  { id: "3", title: "Write README", points: 1, status: "todo" },  
];
```

Board: describe the columns once as data, then map over them and filter the tasks for each status. This avoids repeating three Column lines. Because this is a list, each Column needs a key.

src/components/Board.jsx

```
import Column from "../Column.jsx";
```

```
const columns = [ ["todo", "To do"], ["doing", "In progress"], ["done", "Done"]];
```

```
function Board({ tasks }) {  
  return (  
    {columns.map(([status, heading]) => (  
      <Column  
        key={status}  
        heading={heading}  
        tasks={tasks.filter((t) => t.status === status)}  
      />  
    ))}  
  )  
}
```

```

);
}
export default Board;

src/App.css

.board {
display: grid;
grid-template-columns: repeat(3, 1fr);
gap: 16px;
}

```

Troubleshooting

| Problem | Fix |

| --- | --- |

| Tasks appear in the wrong column, or in no column | The status spelling differs between the data and the filter, for example "in-progress" against "doing". |

| Everything is in one column | The CSS for .board is missing display: grid and grid-template-columns: repeat(3, 1fr). |

| Header count is wrong | You passed a filtered list to Header instead of the full list. |

| Every card shows "Open", or a column is empty | A component still reads done, which no longer exists in the data. Use status. |

> Stretch challenge

> - Add a points total to each column heading using reduce.

> - Add a colour per status using a class name, for example column column-doing.

When you finish, open React DevTools and look at your component tree: App, Header, Board, three Columns and the TaskCards inside them.

Knowledge check

Test yourself on today's content. Try to answer without looking back, then check the [answer key](#answer-key). These are the questions for tomorrow morning's recap.

1. What attribute replaces class in JSX?
2. How does a component receive data from its parent?
3. What is the children prop?
4. Why should list keys not be the array index when items can be deleted?
5. What is wrong with {count} && {count} items when count is 0?

Key takeaways

What you learned

? JSX is JavaScript: `{}` for expressions, `className`, closed tags, camelCase.

? Components are functions; props are their read-only inputs.

? `children` enables flexible wrapper components.

- ? `map` renders lists; keys must be stable and unique.
- ? `&&`, ternaries and early returns control what appears.
- ? Plan components first: the hierarchy, then a static build.

Take-home practice

> Add a Footer component and a SummaryBar that shows the total number of tasks, the total points and the percentage done (tasks whose status is "done"). Calculate everything from the tasks array; do not store any totals.

The important rule is calculate, do not store. It sets up tomorrow's lesson on derived state.

Looking ahead

Today you went from a blank App to a real component tree that renders data. That is the core of React. But TaskBoard cannot do anything yet: you cannot add or complete a task. Tomorrow you fix that with state and events, which is where React becomes interactive.

Further reading

Every topic today has a matching page on react.dev with interactive challenges at the bottom. Doing those challenges is excellent extra practice.

- | Resource | Link |
- | --- | --- |
- | React: Your first component | <https://react.dev/learn/your-first-component> |
- | React: Writing markup with JSX | <https://react.dev/learn/writing-markup-with-jsx> |
- | React: Passing props | <https://react.dev/learn/passing-props-to-a-component> |
- | React: Conditional rendering | <https://react.dev/learn/conditional-rendering> |
- | React: Rendering lists | <https://react.dev/learn/rendering-lists> |
- | React: Thinking in React | <https://react.dev/learn/thinking-in-react> |
- | React Developer Tools | <https://react.dev/learn/react-developer-tools> |

Answer key

Use these answers to check your own work. If an answer surprises you, return to the matching module.

Morning recap: Day 2 knowledge check

- 10.
- { ...task, done: true }
- filter.
- fetch does not reject on HTTP errors such as 404 or 500; res.ok tells you whether the status was 200 to 299.
- src/main.jsx, using createRoot on the #root element.

Knowledge check

- className.
- Through props, passed like attributes and received as one object.
- Whatever is placed between the component's opening and closing tags.

4. Indexes shift when items are removed, so React can mismatch items and their internal state.
5. It renders a stray 0. Use `count > 0` && instead.

My notes

